

Check Point 01

Segurança no código e nas dependências

[njsscan](#), [npm audit](#), [Datree](#) e [StackHawk](#)

Grupo 3

Leonardo Garroti e Aquiles Fonseca

Agenda e tempo

Bloco	Tempo
Abertura e categorias	3 minutos
As quatro ferramentas	10 minutos
Laboratório com a turma	12 minutos
Comparativo e conclusão	3 minutos
Perguntas	2 minutos

Quatro formas de procurar falhas

Categoria	O que analisa	Aplicação no ar?
SAST	Código da equipe	Não
SCA	Dependências e versões	Não
IaC	Configuração de infraestrutura	Não
DAST	Aplicação funcionando	Sim

SAST e SCA no nosso exemplo

SAST

Entrada do usuário chega a eval.

A correção muda o app.js.

SCA

O projeto usa lodash vulnerável.

A correção atualiza a biblioteca e o lockfile.

Pipeline, testes cedo e SBOM

Momento	Teste
Code e Build	SAST e IaC
Build	SCA e lista de componentes
Test e Release	DAST com aplicação funcionando
Antes de Deploy	Conferir a configuração do ambiente

SBOM é a lista de componentes. Testar cedo ajuda, e o DAST ainda tem seu papel.

njsscan: SAST para Node.js

Analisa o código sem subir o servidor.

Ajin Abraham. Python. Licença LGPL-3.0.

Versão usada: 1.0.0, com Semgrep 1.172.0.

Combina padrões de texto e regras sobre a estrutura do código.

njsscan: o que encontramos

Regra	Tipo da falha	Nível
express_xss	CWE-79, XSS	ERROR
generic_os_command_exec	CWE-78, comando	ERROR
eval_nodejs	CWE-95, eval	ERROR

Três ocorrências reais na versão insegura. O gate do grupo trata ERROR como HIGH.

Cuidados com o njsscan

Um relatório vazio exige conferir se a análise rodou.

No Windows, libsast pula a etapa semântica. Usamos uma ponte para chamar o Semgrep.

No Docker e no GitHub Actions, a execução usa Linux.

As regras têm limites. A revisão do alerta continua necessária.

npm audit: SCA das dependências

Comando do npm, disponível desde npm 6, em 2018.

Consulta avisos conhecidos para versões dos pacotes.

CLI em JavaScript. Licença principal Artistic License 2.0.

Versão usada: npm 11.12.0. Não precisa do servidor no ar.

npm audit: um pacote, vários avisos

Versão do lodash	Resultado local
4.17.15	1 pacote HIGH, com vários avisos
4.18.1	0 pacotes vulneráveis

CVE-2020-8203 aparece no relatório, com CVSS 7,4. A presença foi confirmada, sem provar exploração na rota chunk.

Datree: políticas para Kubernetes

Lê manifests e compara a configuração com políticas.

Projeto em Go, com licença Apache-2.0.

Pode verificar YAML, esquema e regras da configuração.

Não precisa de uma aplicação web funcionando.

Exemplo de configuração de risco

privileged: true

runAsUser: 0

hostNetwork: true

image: nginx:latest

Mais acesso para o container

Versão de imagem menos
previsível

Manifest próprio para discussão. Não aplicado a um cluster e não analisado por Datree.

Datree: manutenção encerrada

Projeto arquivado

Empresa encerrada em julho de 2023.

Repositório arquivado em junho de 2024.

Última release: 1.9.19, de julho de 2023.

Mantivemos o nome da tabela e registramos o limite.

StackHawk: DAST com HawkScan

Envia requisições para a aplicação funcionando.

Precisa alcançar as rotas e, às vezes, usar autenticação.

HawkScan faz a análise. A plataforma organiza os resultados.

Testes ativos precisam de um alvo próprio e ambiente de teste.

StackHawk: acesso e limites

Produto comercial, com origem no ZAP e motor próprio HSTE.

A execução precisa de configuração, conta e chave.

A CLI permite salvar resultados em SARIF.

Ferramenta pesquisada, sem execução DAST neste trabalho.

Laboratório: preparação

Imagem pronta antes da aula

```
docker pull node:24-bookworm-slim  
docker compose build --pull lab
```

Aplicação própria, sem portas publicadas. O npm audit precisa de internet para consultar o registro.

Versão insegura: gate bloqueado

`docker compose run --rm lab inseguro`

njsscan: 3 ERROR.

npm audit: 1 pacote HIGH.

Gate local: saída 1.

CHECK POINT 01 / GRUPO 3 / EVIDÊNCIA 04

Gate: teste bloqueado

Versão insegura

BLOQUEADO

Teste	Resultado
njsscan	3 ERROR, tratados como HIGH
npm audit	1 pacote HIGH
Código de saída do gate	1

Há falhas de nível alto. O gate impede a aprovação desta versão.

Leonardo Garroli e Aquiles Fonseca
Execução local em 2026-09-16. Tela criada a partir dos relatórios reais, sem simular o GitHub Actions.

Leitura dos três achados

Entrada chega a	CWE	Correção
Resposta HTML	79	Devolver texto simples
exec	78	Retirar execução de comando
eval	95	Somar números validados

Três verdadeiros positivos na amostra revisada. A amostra é pequena e tem falhas intencionais.

Mudanças na versão corrigida

Texto simples no lugar de HTML com entrada.

Mensagem fixa no lugar de comando recebido.

Soma validada no lugar de eval.

Lodash atualizado para 4.18.1.

CHECK POINT 01 / GRUPO 3 / EVIDÊNCIA 05

Correções aplicadas

Antes	Depois
Nome dentro de uma resposta HTML	Resposta como texto simples
exec com comando enviado pelo usuário	Resposta fixa, sem executar comando
eval com expressão enviada pelo usuário	Soma de dois números validados
lodash 4.17.15	lodash 4.18.1

As correções mudam o código e a versão da dependência.

Nenhum alerta foi escondido com comentário de exclusão ou redução de severidade.

Leonardo Garroli e Aquiles Fonseca
Execução local em 2026-09-16. Tela criada a partir dos relatórios reais, sem simular o GitHub Actions.

Versão corrigida: gate aprovado

`docker compose run --rm lab corrigido`

`njsscan: 0 alertas.`

`npm audit: 0 pacotes vulneráveis.`

Gate local: saída 0.

CHECK POINT 01 / GRUPO 3 / EVIDÊNCIA 06

Gate: teste aprovado

Versão corrigida

APROVADO

Teste	Resultado
njsscan	0 alertas
npm audit	0 pacotes vulneráveis
Código de saída do gate	0

Os dois scanners passaram no alvo corrigido.

Esse resultado cobre os testes feitos. Ele não prova que a aplicação esteja livre de qualquer falha.

Leonardo Garroli e Aguilas Fonseca
Execução local em 2026-09-16. Tela criada a partir dos relatórios reais, sem simular o GitHub Actions.

Perguntas para a turma

1. Quantas ocorrências ERROR o njsscan mostrou na versão insegura? Como o nosso gate trata esse nível?
2. Qual CWE aparece no uso de eval? Qual mudança retirou essa falha?

Entrega individual: print da própria execução e respostas.

Comparativo das quatro ferramentas

Ferramenta	Ponto útil	Limite do nosso estudo
njsscan	Liga regra ao código	Alvo pequeno e ponte no Windows
npm audit	Consulta simples de avisos	Depende da base e da rede
Datree	Políticas Kubernetes	Projeto arquivado, não executado
StackHawk	Aplicação e APIs em execução	Produto comercial, não executado

Conclusão do trabalho

Mudar o código resolve os alertas SAST. Atualizar a biblioteca resolve o alerta SCA.

Verificar cedo ajuda a equipe. IaC e DAST completam a cobertura.

Antes da aula: teste Docker do zero, confira os builds e ensaie em dupla.

IA ajudou na preparação. Os limites e as fontes estão declarados.

Referências e perguntas

Documentação oficial: njsscan, npm, Datree e StackHawk.

OWASP Top 10:2025 e NIST SSDF.

MITRE CWE e GitHub Advisory Database.

Referências completas no PDF e em REFERENCIAS.md.

Leonardo Garroti e Aquiles Fonseca